CTPIP

Architecture and Technology Overview

Document control	
Document ID	CTPIP-AO-001
Project	Clinical Trial Protocol Intelligence Platform (CTPIP)
Version	1.1
Date	21 October 2026
Status	Released

Version	Date	Description of change
1.1	21 October 2026	Initial formal release.

Summary

- Purpose and scope of the platform.
- System architecture and runtime view.
- Technology stack and dependencies.
- Pipelines in brief, design principles and current gaps.

Table of Contents

1. Purpose . . . 3

2. System Architecture . . . 3

3. Technology Stack . . . 4

4. Pipelines in Brief . . . 4

4.1 Ingestion (PDF to graph) . . . 4

4.2 Query (question to answer) . . . 5

4.3 Local AI (explain, ask, agent) . . . 5

4.4 Details . . . 5

5. Runtime View . . . 5

6. Design Principles . . . 5

7. Current State and Known Gaps . . . 6

1. Purpose

- CTPIP reads clinical trial protocol PDFs and builds a searchable, evidence-linked knowledge graph.
- Users can ask plain-English questions, explore the graph, search wording and compare protocols.
- Current content: 187 breast-cancer protocols, 19,413 pages, 95,181 evidence passages, 29,047 clinical facts.
- Every fact is traceable to the exact PDF page and quote; nothing is invented.
- A local AI assistant (Ollama) explains protocols and results in plain language and answers questions with cited sources.
- Runs fully locally; the AI is optional and every other feature works without it.

2. System Architecture

- Three layers: user interface, engine (backend), storage.
- A background worker keeps the website responsive while PDFs are processed.

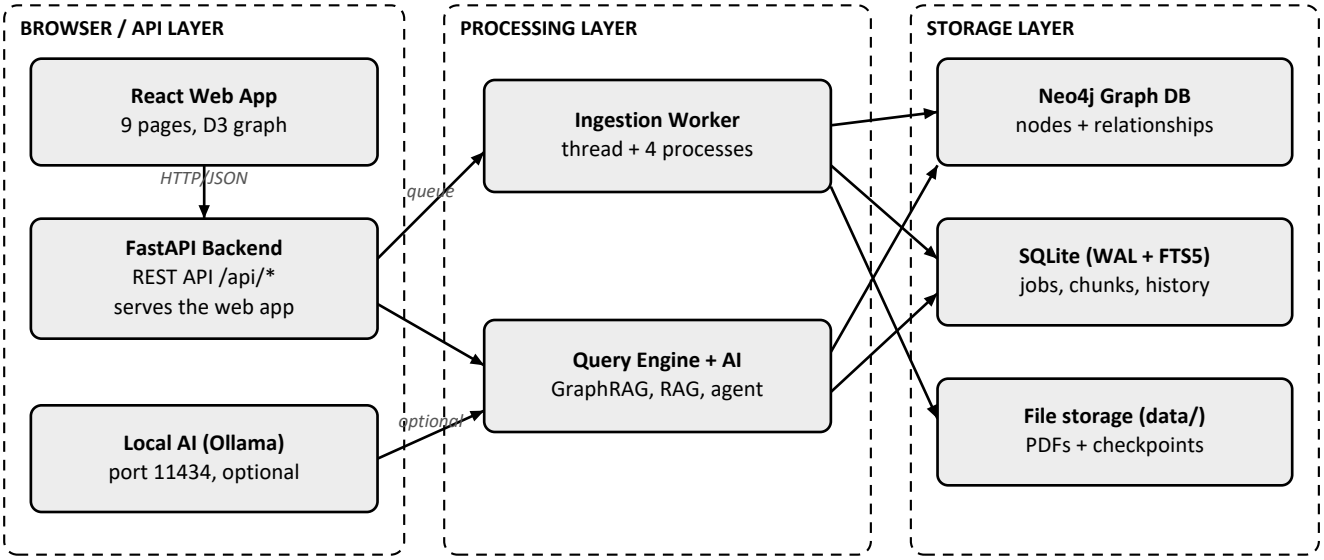


Figure 1: Components and their connections.

Component	Role	Code location
React web app	Screens: upload, query, graph, compare, quality, history.	frontend/src
FastAPI backend	REST API; serves the built web app on port 8000.	backend/app/main.py, api/
Ingestion worker	Processes PDFs in parallel with checkpoints.	backend/app/ingest/
Neo4j	Knowledge graph; read-only queries for questions.	backend/app/graph/

Component	Role	Code location
SQLite	Documents, jobs, passages (full-text), warnings, history, audit.	backend/app/core/metadb.py
Artifacts (.json.gz)	Per-document checkpoints that allow resume and re-graph.	data/artifacts/
Query engine	Plans, runs, validates and summarises questions.	backend/app/query/
Local AI assistant	Plain-language explanations, protocol study, RAG chat, research agent and grounding check; talks to Ollama.	backend/app/llm/, api/assistant.py

3. Technology Stack

Layer	Technology	Purpose
Frontend	React 18.3, TypeScript 5.6, Vite 5.4	Interface and build.
Frontend	React Router 6.28, TanStack Query 5.62	Navigation, data fetching.
Frontend	D3 7.9, Tailwind CSS 3.4, lucide-react	Graph drawing, styling, icons.
Backend	Python 3.11+, FastAPI >= 0.115, Uvicorn >= 0.30	API and server.
Backend	Pydantic >= 2.7, pydantic-settings, python-multipart, httpx, pytest	Validation, settings, upload, AI calls, tests.
Graph	Neo4j 5 (driver >= 5.28, < 6), Cypher	Persistent graph.
Storage	SQLite (WAL, FTS5), gzip JSON	Metadata, search, checkpoints.
PDF	PyMuPDF >= 1.24, RapidOCR (onnxruntime) >= 1.3	Text, tables, page images, OCR.
Language	Curated lexicon, regular expressions	Entities and facts.
Local AI	Ollama (native API), default model qwen2.5:3b; any OpenAI-compatible server	Explanations, Q&A;, agent (optional).
Infrastructure	start.bat, Java 21 (Neo4j Desktop)	Launcher (Neo4j, Ollama, free port) and Neo4j runtime.

4. Pipelines in Brief

4.1 Ingestion (PDF to graph)

- 1 Register: validate the PDF, SHA-256 duplicate check, store a copy, queue a job.
- 2 Extract: text, fonts, positions and tables with PyMuPDF; OCR for scans.
- 3 Structure: headings from outline or layout; classify sections.
- 4 Chunk: split into passages that keep page and position.

- 5 Extract facts: dictionary entities and rule-based claims with quotes.
- 6 Store: sections, chunks and search index in SQLite.
- 7 Resolve and write graph: entity resolution, schema validation, Neo4j write with provenance.
- Progress is saved after each stage; failures retry up to 3 times and resume from the checkpoint.

4.2 Query (question to answer)

- Parse intent, link terms to graph entities, plan allow-listed templates.
- Run read-only parameterised Cypher with timeout and result limit.
- Collect nodes, edges and paths; fetch claims and passages; rank by confidence, evidence and path length.
- Build BFS or DFS answer tree; validate provenance; write evidence-only answer; store in history.

4.3 Local AI (explain, ask, agent)

- Retrieval: full-text search with synonym expansion and section-aware ranking, plus validated graph facts and the protocol's structured facts; each source records why it was chosen.
- Generation: a clinical-trial protocol expert prompt; the model may use only the numbered sources and must cite them; answers stream to the browser.
- Research agent: the model plans up to four tool calls (find protocols, protocol overview, text search, graph search, compare); each step is shown.
- Explainability check: cited statements, automatically matched statements, invalid citations and numbers not found in the sources; verdict well, partly or weakly grounded.

4.4 Details

- See `03_Pipeline_Explanation.pdf` and `05_Pipeline.pptx`.

5. Runtime View

- start.bat starts Neo4j (port 7687), then the backend (uvicorn, port 8000); the browser connects to the backend.
- Backend threads: request threads, ingest dispatcher, writer; process pool of 4 for PDF analysis.
- Start-up: create SQLite schema and AI tables, create Neo4j constraints and indexes, warm the dictionary matcher, start the worker; on an empty store, load the dataset automatically.
- Ollama runs as a separate local service on port 11434 (started by start.bat when installed).
- If Neo4j is down the API still serves the library and history; graph endpoints return HTTP 503.

6. Design Principles

- Evidence first: every clinical relationship has claim id, document, chunk, page, quote and confidence.

- Deterministic by default: same input gives same output; offline.
- AI as untrusted helper: accepted only when schema-valid and quote-verified.
- No raw queries: allow-listed read-only templates with timeouts, limits and an optional reader account.
- Never silently merge: look-alike drug names go to human review.
- Idempotent and restartable: re-processing never duplicates; deleting a document removes all its data.

7. Current State and Known Gaps

- All 187 documents are processed.
- `api/evaluation.py` imports `app.evaluation.runner`, which is not present, so the evaluation endpoint fails.
- `backend/tests/` contains unit tests for the AI explainability helpers; other modules have no automated tests yet.
- The lexicon is a curated breast-oncology dictionary, not UMLS, RxNorm or MeSH.
- Some stored metadata is imperfect (for example PALOMA-3 is stored as Phase 2 while its title says Phase 3); quotes beside each fact make this visible.
- Small local models cite less reliably than larger ones; the grounding check and automatic source matching compensate.
- `data/` and `.env` are not part of the public repository; the database and graph are rebuilt on first start.